

Web Scraping

Lecture 13

Dr. Colin Rundel



Hypertext Markup Language

Most of the data on the web is still largely available as HTML - while it is structured (hierarchical) it often is not available in a form useful for analysis (flat / tidy).

```
1 <html>
2   <head>
3     <title>This is a title</title>
4   </head>
5   <body>
6     <p align="center">Hello world!</p>
7     <br/>
8     <div class="name" id="first">John</div>
9     <div class="name" id="last">Doe</div>
10    <div class="contact">
11      <div class="home">555-555-1234</div>
12      <div class="home">555-555-2345</div>
13      <div class="work">555-555-9999</div>
14      <div class="fax">555-555-8888</div>
15    </div>
16  </body>
17 </html>
```

rvest

`rvest` is a package from the tidyverse that makes basic processing and manipulation of HTML data straightforward. It provides high level functions for interacting with html via the `xml2` library.

Core functions:

- `read_html()` - read HTML data from a url or character string.
- `html_elements()` / `html_nodes()` - select specified elements from the HTML document using CSS selectors (or xpath).
- `html_element()` / `html_node()` - select a single element from the HTML document using CSS selectors (or xpath).
- `html_table()` - parse an HTML table into a data frame.
- `html_text()` / `html_text2()` - extract tag's text content.
- `html_name()` - extract a tag/element's name(s).
- `html_attrs()` - extract all attributes.
- `html_attr()` - extract attribute value(s) by name.

html, rvest, & xml2

```
1 html =  
2 '<html>  
3   <head>  
4     <title>This is a title</title>  
5   </head>  
6   <body>  
7     <p align="center">Hello world!</p>  
8     <br/>  
9     <div class="name" id="first">John</div>  
10    <div class="name" id="last">Doe</div>  
11    <div class="contact">  
12      <div class="home">555-555-1234</div>  
13      <div class="home">555-555-2345</div>  
14      <div class="work">555-555-9999</div>  
15      <div class="fax">555-555-8888</div>  
16    </div>  
17  </body>  
18 </html>'
```

```
1 read_html(html)
```

```
{html_document}
```

```
<html>
```

```
[1] <head>\n<meta http-equiv="Content-Type" cont
```

```
[2] <body>\n    <p align="center">Hello world!</
```

Selecting elements

```
1 read_html(html) |> html_elements("p")
```

```
{xml_node_set (1)}  
[1] <p align="center">Hello world!</p>
```

```
1 read_html(html) |> html_elements("p") |> html_text()
```

```
[1] "Hello world!"
```

```
1 read_html(html) |> html_elements("p") |> html_name()
```

```
[1] "p"
```

```
1 read_html(html) |> html_elements("p") |> html_attrs()
```

```
[[1]]  
  align  
"center"
```

```
1 read_html(html) |> html_elements("p") |> html_attr("align")
```

```
[1] "center"
```

More selecting tags

```
1 read_html(html) |> html_elements("div")
```

```
{xml_nodeset (7)}  
[1] <div class="name" id="first">John</div>  
[2] <div class="name" id="last">Doe</div>  
[3] <div class="contact">\n      <div class="home">555-555-1234</div>\n      ...  
[4] <div class="home">555-555-1234</div>  
[5] <div class="home">555-555-2345</div>  
[6] <div class="work">555-555-9999</div>  
[7] <div class="fax">555-555-8888</div>
```

```
1 read_html(html) |> html_elements("div") |> html_text()
```

```
[1] "John"  
[2] "Doe"  
[3] "\n      555-555-1234\n      555-555-2345\n      555-555-9999\n      555-555-8888\  
[4] "555-555-1234"  
[5] "555-555-2345"  
[6] "555-555-9999"  
[7] "555-555-8888"
```

Nesting tags

```
1 read_html(html) |> html_elements("body div")
```

```
{xml_nodeset (7)}  
[1] <div class="name" id="first">John</div>  
[2] <div class="name" id="last">Doe</div>  
[3] <div class="contact">\n      <div class="home">555-555-1234</div>\n      ...  
[4] <div class="home">555-555-1234</div>  
[5] <div class="home">555-555-2345</div>  
[6] <div class="work">555-555-9999</div>  
[7] <div class="fax">555-555-8888</div>
```

```
1 read_html(html) |> html_elements("body>div")
```

```
{xml_nodeset (3)}  
[1] <div class="name" id="first">John</div>  
[2] <div class="name" id="last">Doe</div>  
[3] <div class="contact">\n      <div class="home">555-555-1234</div>\n      ...
```



```
1 read_html(html) |> html_elements("body div div")
```

```
{xml_nodeset (4)}
```

```
[1] <div class="home">555-555-1234</div>
```

```
[2] <div class="home">555-555-2345</div>
```

```
[3] <div class="work">555-555-9999</div>
```

```
[4] <div class="fax">555-555-8888</div>
```

css selectors

We will be using a tool called selector gadget to help us identify the html elements of interest - it does this by constructing a css selector which can be used to subset the html document.

Some examples of basic selector syntax is below,

Selector	Example	Description
.class	<code>.title</code>	Select all elements with class="title"
#id	<code>#name</code>	Select all elements with id="name"
element	<code>p</code>	Select all <p> elements
element element	<code>div p</code>	Select all <p> elements inside a <div> element
element>element	<code>div > p</code>	Select all <p> elements with <div> as a parent
[attribute]	<code>[class]</code>	Select all elements with a class attribute
[attribute=value]	<code>[class=title]</code>	Select all elements with class="title"

There are also a number of additional combinators and pseudo-classes that improve flexibility, see examples [here](#).

CSS classes and ids

```
1 read_html(html) |> html_elements(".name")
```

```
{xml_nodeset (2)}
```

```
[1] <div class="name" id="first">John</div>
```

```
[2] <div class="name" id="last">Doe</div>
```

```
1 read_html(html) |> html_elements("div.name")
```

```
{xml_nodeset (2)}
```

```
[1] <div class="name" id="first">John</div>
```

```
[2] <div class="name" id="last">Doe</div>
```

```
1 read_html(html) |> html_elements("#first")
```

```
{xml_nodeset (1)}
```

```
[1] <div class="name" id="first">John</div>
```

Mixing it up

```
1 read_html(html) |> html_elements("[align]")
```

```
{xml_nodeset (1)}  
[1] <p align="center">Hello world!</p>
```

```
1 read_html(html) |> html_elements(".contact div")
```

```
{xml_nodeset (4)}  
[1] <div class="home">555-555-1234</div>  
[2] <div class="home">555-555-2345</div>  
[3] <div class="work">555-555-9999</div>  
[4] <div class="fax">555-555-8888</div>
```

html_text() vs html_text2()

```
1 html = read_html(  
2     "<p>  
3     This is the first sentence in the paragraph.  
4     This is the second sentence that should be on the same line as the first sentence  
5     </p>"  
6 )
```

```
1 html |> html_text()
```

```
[1] "  \n    This is the first sentence in the paragraph.\n    This is the second sentence that should be on the same line as the first sentence"
```

```
1 html |> html_text2()
```

```
[1] "This is the first sentence in the paragraph. This is the second sentence that should be on the same line as the first sentence"
```

```
1 html |> html_text() |> cat(sep="\n")
```

This is the first sentence in the paragraph.

This is the second sentence that should be on the same line :

```
1 html |> html_text2() |> cat(sep="\n")
```

This is the first sentence in the paragraph. This is the second :

This third sentence should start on a new line.

html tables

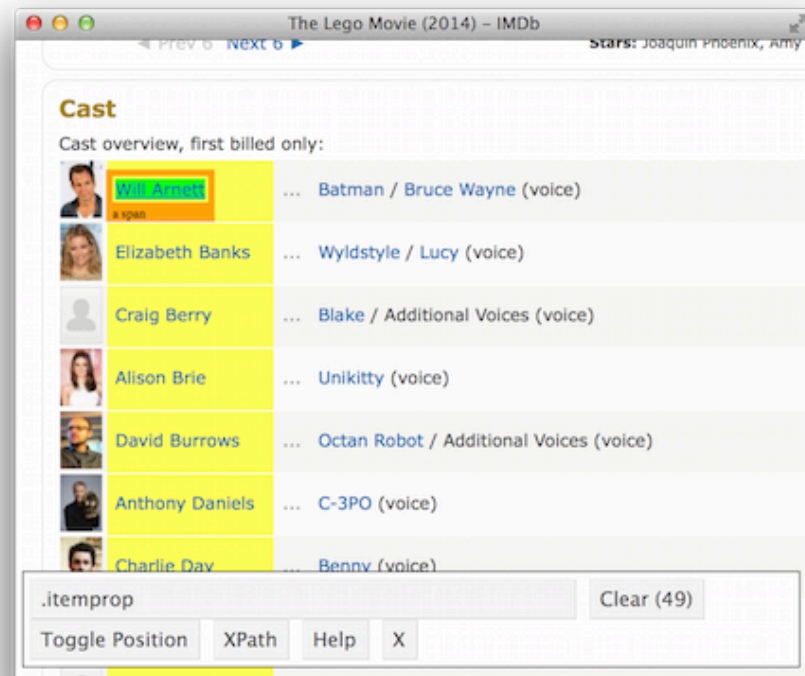
```
1 html_table =  
2 '<html>  
3   <head>  
4     <title>This is a title</title>  
5   </head>  
6   <body>  
7     <table>  
8       <tr> <th>a</th> <th>b</th> <th>c</th> </tr>  
9       <tr> <td>1</td> <td>2</td> <td>3</td> </tr>  
10      <tr> <td>2</td> <td>3</td> <td>4</td> </tr>  
11      <tr> <td>3</td> <td>4</td> <td>5</td> </tr>  
12    </table>  
13  </body>  
14 </html>'
```

```
1 read_html(html_table) |>  
2   html_elements("table") |>  
3   html_table()
```

```
[[1]]  
# A tibble: 3 × 3  
      a     b     c  
  <int> <int> <int>  
1     1     2     3  
2     2     3     4  
3     3     4     5
```

SelectorGadget

This is a javascript based tool that helps you interactively build an appropriate CSS selector for the content you are interested in.



selectorgadget.com

Sta 323 - Spring 2026

Web scraping considerations

“Can you?” vs “Should you?”

Researchers just released profile data on 70,000 OkCupid users without permission

By Brian Resnick | @B_resnick | brian@vox.com | May 12, 2016, 6:00pm EDT

A group of researchers has released a data set on nearly 70,000 users of the online dating site OkCupid. The data dump breaks the cardinal rule of social science research ethics: **It took identifiable personal data without permission.**

The information — while publicly available to OkCupid users — was collected by Danish researchers who never contacted OkCupid or its clientele about using it.

The data, collected from November 2014 to March 2015, includes user names, ages, gender, religion, and personality traits, as well as answers to the personal questions the site asks to help match potential mates. The users hail from a few dozen countries around the world.

The data dump did not reveal anyone's real name. But it's entirely possible to use clues from a user's location, demographics, and OkCupid user name to determine their identity.

If your OkC username is one you've used anywhere else, I now know your sexual preferences & kinks, your answers to thousands of questions.

— Scott B. Weingart (@scott_bot) May 11, 2016

“Can you?” vs “Should you?”



Emil OW Kirkegaard @KirkegaardEmil · May 8

The OKCupid paper has now been submitted. This means that the dataset is now public! Enjoy! :) [openpsych.net/forum/showthre...](https://openpsych.net/forum/showthread.php?p=10000)



26



38



Ethan Jewett @esjewett · May 11

@KirkegaardEmil This data set is highly re-identifiable. Even includes usernames? Was any work at all done to anonymize it?



3



9



Emil OW Kirkegaard

@KirkegaardEmil



Follow

@esjewett No. Data is already public.

Scraping permission & robots.txt

There is a standard for communicating to users if it is acceptable to automatically scrape a website via the [robots exclusion standard](#) or [robots.txt](#).

You can find examples at all of your favorite websites: [google](#), [facebook](#), etc.

These files are meant to be machine readable, but the [polite](#) package can handle this for us (and much more).

```
1 polite::bow("http://google.com")
```

```
<polite session> http://google.com
  User-agent: polite R package
  robots.txt: 473 rules are defined for 5 bots
  Crawl delay: 5 sec
  The path is scrapable for this user-agent
```

```
1 polite::bow("http://facebook.com")
```

```
<polite session> http://facebook.com
  User-agent: polite R package
  robots.txt: 720 rules are defined for 34 bots
  Crawl delay: 5 sec
  The path is not scrapable for this user-agent
```

Scraping with polite

Beyond the `bow()` function, `polite` also has a `scrape()` function that helps you scrape a website while maintaining the three pillars of a polite session:

- seek permission,
- take it slowly
- never ask twice.

This is achieved by using the session object from `bow()` within the `scrape()` function to make the request (this is equivalent to `rvest`'s `read_html()` and returns a parsed html object).

New paths within the website can be accessed by using the `nod()` function before using `scrape()`.

Rate limiting

Making requests too quickly can overload a server, get your IP blocked, or violate a site's terms of service. Adding delays between requests is essential for responsible scraping.

The simplest approach is `Sys.sleep()` between requests:

```
1 for (url in urls) {  
2   read_html(url) |> html_elements(".title")  
3   Sys.sleep(1) # wait 1 second between requests  
4 }
```

`polite` handles this automatically — `scrape()` respects the crawl delay specified in `robots.txt` and defaults to a 5-second delay if none is specified:

```
1 session = polite::bow("https://example.com")  
2  
3 polite::nod(session, "page/1") |> polite::scrape()  
4 Sys.sleep(1) # polite already delays, but you can add more  
5 polite::nod(session, "page/2") |> polite::scrape()
```

JavaScript rendered pages

`read_html()` only sees the static HTML delivered by the server — pages that use JavaScript to render content will appear empty or incomplete.

`rvest::read_html_live()` handles this by running a headless Chrome browser (via the `chromote` package) that fully renders the page before returning the HTML:

```
1 page = read_html_live("https://example.com")
2
3 page |> html_elements(".some-js-rendered-class")
```

The returned object also supports interacting with the page like a user would:

```
1 page$click("#load-more-button")
2 page$scroll_to(top = 1000)
3 page$type("#search-box", "query text")
```

Graceful error handling

When scraping many pages, individual requests will often fail (missing elements, network errors, changed page structure, etc.). Rather than stopping entirely, we can handle errors gracefully.

`purrr::possibly()` wraps a function so that errors return a default value instead of stopping:

```
1 safe_read = purrr::possibly(read_html, otherwise = NULL)
2
3 urls |>
4   purrr::map(safe_read)
```

`purrr::safely()` is similar but returns a list with `result` and `error` components, letting you inspect what went wrong:

```
1 safe_read = purrr::safely(read_html)
2
3 results = urls |> purrr::map(safe_read)
4
5 results |> purrr::map("result") # successful results (NULL on failure)
6 results |> purrr::map("error")  # error messages (NULL on success)
```


Example - Rotten Tomatoes

For the movies listed in the **Popular Streaming Movies** list on [rottentomatoes.com](https://www.rottentomatoes.com) create a data frame with the movies' titles, their tomatometer score, and whether the movie is fresh or rotten, and the movie's url.

Exercise 1

Using the url for each movie, now go out and grab the mpaa rating, the runtime and number of user ratings.

If you finish that you can then try to scrape the Tomatometer and audience scores for each movie.